

ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І. І. МЕЧНИКОВА
ФАКУЛЬТЕТ МАТЕМАТИКИ, ФІЗИКИ І ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА ОПТИМАЛЬНОГО КЕРУВАННЯ І ЕКОНОМІЧНОЇ КІБЕРНЕТИКИ

Паралельні алгоритми обчислювальної математики

Звіт **з лабораторної роботи №1** **Технологія OpenMP. Загальні та приватні змінні**

Виконав: студент 1 курсу магістратури
спеціальності 113 Прикладна математика
Чачко Натан Леонідович

2025 р.

Завдання

Число π з заданою точністю можна обчислити таким чином. Оскільки,

$$\pi = \int_0^1 \frac{4dx}{1+x^2} \quad (1)$$

то обчислюючи визначений інтеграл

$$I(f) = \int_a^b f(x)dx$$

наприклад, за складеною формулою середніх прямокутників

$$S_h(f) = h \sum_{i=0}^{n-1} f(x_{i-1/2})$$

маємо наближене значення числа π . Тут $f(x) = \frac{4}{1+x^2}$, n — ціле число, яке визначає крок складеної квадратурної формули, $h = 1/n$, $x_{i-1/2} = h(i - 1/2)$. Чим n більше, тим точніше обчислюється інтеграл (1), а значить, точніше обчислюється число π . Нижче наведено послідовну програму обчислення числа π за допомогою чисельного інтегрування.

```
1 #include <stdio.h>
2 // Обчислення числа pi
3 double f(double x) {return(4.0/(1.0+x*x));}
4 int main()
5 {
6     double x, sum, pi;
7     long int i,n = 100000;
8     h = 1.0/n;
9     sum = 0.0;
10    for(i=0; i < n; i++)
11    {
12        x = h*(i-0.5);
13        sum = sum + f(x);
14    }
15    pi = h*sum;
16    printf("pi = %f\n", pi);
17 }
```

Використовуючи технологію OpenMP, написати паралельну версію цієї програми. Провести обчислювальний експеримент та заповнити таблицю 1.

Побудувати графік залежності точності обчислення числа π від числа n .

Побудувати графіки залежності часу обчислення числа π від кількості потоків для різних n .

До звіту включити відомості про обчислювальну систему та компілятор, що використовувались для обчислень.

```

1 #include <stdio.h>
2 #include <omp.h>
3 #include <math.h>
4
5 double f(const double y) {
6     return 4.0 / (1.0 + y * y);
7 }
8
9 void calculate(const int power) {
10     double x;
11     long long i;
12     const long long n = pow(10, power); // Кількість розбиттів (ітерацій)
13     printf("\nn = %lld\n", n);
14     const double h = 1.0 / n; // Ширина прямокутника (крок інтегрування)
15     double sum = 0.0;
16     // === ПОСЛІДОВНЕ ОБЧИСЛЕННЯ ===
17     double s_time = omp_get_wtime();
18     for (i = 1; i <= n; i++) {
19         x = h * (i - 0.5); // Центр прямокутника
20         sum += f(x); // Додавання площі прямокутника до суми
21     }
22     double pi = h * sum; // Остаточне наближення до числа pi
23     double f_time = omp_get_wtime();
24     printf("[SEQUENTIAL] pi = %.5f | error = %.15f | time = %.4f sec\n", pi, fabs(M_PI -
25         pi), f_time - s_time);
26
27     // === ПАРАЛЕЛЬНЕ ОБЧИСЛЕННЯ з 2 потоками ===
28     s_time = omp_get_wtime();
29     sum = 0.0;
30     #pragma omp parallel for private(x) shared(h) reduction(+:sum) num_threads(2)
31     for (i = 1; i <= n; i++) {
32         x = h * (i - 0.5); // Центр прямокутника
33         sum += f(x); // Додавання в потоці до спільної суми (через reduction)
34     }
35     pi = h * sum;
36     f_time = omp_get_wtime();
37     printf("[PARALLEL 2 THREADS] pi = %.5f | error = %.15f | time = %.4f sec\n", pi, fabs(
38         M_PI - pi), f_time - s_time);
39
40     // === ПАРАЛЕЛЬНЕ ОБЧИСЛЕННЯ з 4 потоками ===
41     s_time = omp_get_wtime();
42     sum = 0.0;
43     #pragma omp parallel for private(x) shared(h) reduction(+:sum) num_threads(4)
44     for (i = 1; i <= n; i++) {
45         x = h * (i - 0.5);
46         sum += f(x);
47     }
48     pi = h * sum;
49     f_time = omp_get_wtime();
50     printf("[PARALLEL 4 THREADS] pi = %.5f | error = %.15f | time = %.4f sec\n", pi, fabs(
51         M_PI - pi), f_time - s_time);
52
53     // === ПАРАЛЕЛЬНЕ ОБЧИСЛЕННЯ з 8 потоками ===
54     s_time = omp_get_wtime();
55     sum = 0.0;
56     #pragma omp parallel for private(x) shared(h) reduction(+:sum) num_threads(8)
57     for (i = 1; i <= n; i++) {
58         x = h * (i - 0.5);
59         sum += f(x);
60     }
61     pi = h * sum;

```

```

59     f_time = omp_get_wtime();
60     printf("[PARALLEL 8 THREADS] pi = %.5f | error = %.15f | time = %.4f sec\n", pi, fabs(
        M_PI - pi), f_time - s_time);
61 }
62
63 int main() {
64     calculate(4);
65     calculate(6);
66     calculate(8);
67     calculate(10);
68     return 0;
69 }

```

Вивід програми:

```

n = 10000
[SEQUENTIAL] pi = 3.14159 | error = 0.000000000833341 | time = 0.0000 sec
[PARALLEL 2 THREADS] pi = 3.14159 | error = 0.000000000833336 | time = 0.0010 sec
[PARALLEL 4 THREADS] pi = 3.14159 | error = 0.000000000833331 | time = 0.0000 sec
[PARALLEL 8 THREADS] pi = 3.14159 | error = 0.000000000833333 | time = 0.0010 sec

n = 1000000
[SEQUENTIAL] pi = 3.14159 | error = 0.000000000000029 | time = 0.0030 sec
[PARALLEL 2 THREADS] pi = 3.14159 | error = 0.000000000000106 | time = 0.0010 sec
[PARALLEL 4 THREADS] pi = 3.14159 | error = 0.000000000000083 | time = 0.0020 sec
[PARALLEL 8 THREADS] pi = 3.14159 | error = 0.000000000000078 | time = 0.0020 sec

n = 100000000
[SEQUENTIAL] pi = 3.14159 | error = 0.0000000000000633 | time = 0.2960 sec
[PARALLEL 2 THREADS] pi = 3.14159 | error = 0.000000000000117 | time = 0.1540 sec
[PARALLEL 4 THREADS] pi = 3.14159 | error = 0.000000000000111 | time = 0.1330 sec
[PARALLEL 8 THREADS] pi = 3.14159 | error = 0.000000000000022 | time = 0.1030 sec

n = 10000000000
[SEQUENTIAL] pi = 3.14159 | error = 0.0000000000000342 | time = 30.4580 sec
[PARALLEL 2 THREADS] pi = 3.14159 | error = 0.000000000000230 | time = 16.5650 sec
[PARALLEL 4 THREADS] pi = 3.14159 | error = 0.000000000000217 | time = 10.7690 sec
[PARALLEL 8 THREADS] pi = 3.14159 | error = 0.000000000000102 | time = 9.6270 sec

```

n	$ \pi - S_h(f) $	1	2	4	8
10^4	0.000000000833333	0.0000	0.0010	0.0000	0.0010
10^6	0.000000000000083	0.0030	0.0010	0.0020	0.0020
10^8	0.000000000000085	0.2960	0.1540	0.1330	0.1030
10^{10}	0.000000000000160	30.4580	16.5650	10.7690	9.6270

Табл. 1: Час обчислення числа π паралельною програмою з різною кількістю потоків.

Пояснення похибки

Хоча математично результат обчислення числа π повинен бути однаковим при будь-якій кількості потоків, на практиці спостерігаються незначні відмінності у похибці. Це пов'язано з особливостями обчислень у паралельних системах:

- **Невизначений порядок операцій при використанні директиви `reduction(+:sum)`.** Кожен потік обчислює часткову суму, після чого OpenMP виконує зведення результатів у довільному порядку.
- **Неасоціативність операції додавання чисел з плаваючою крапкою.** У комп'ютерній арифметиці $(a + b) + c \neq a + (b + c)$ через накопичення похибок округлення.
- **Мікрорізниця у результатах залежно від кількості потоків виникає через різні шляхи обчислень та точність арифметики.** Результати, отримані при 1, 2, 4 чи 8 потоках, обчислюються з тією ж математичною точністю, однак розходження у молодших розрядах неминуче.

Отже, відмінності у похибці для одного і того ж значення n при різних кількості потоків є нормальним явищем і пояснюються властивостями комп'ютерної арифметики.

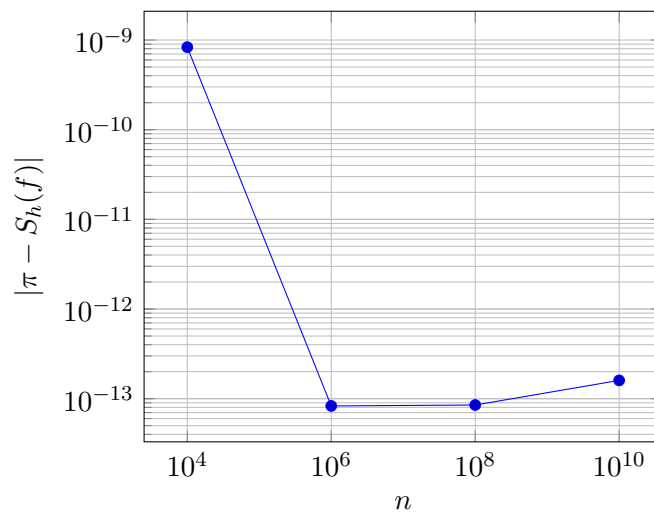


Рис. 1: Залежність точності обчислення числа π від n

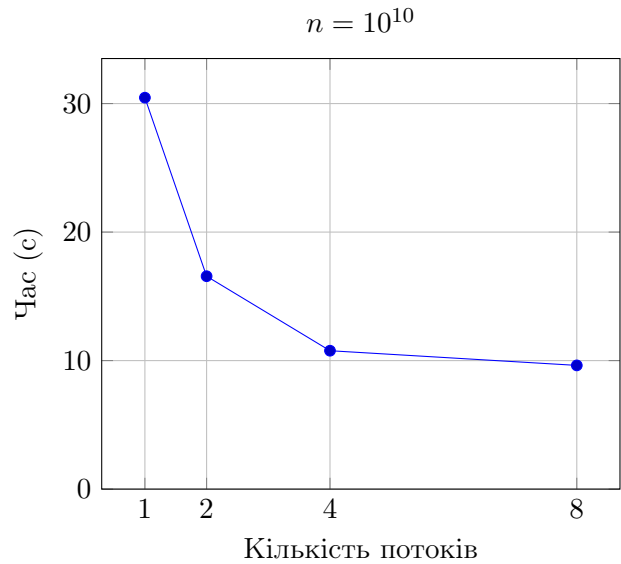
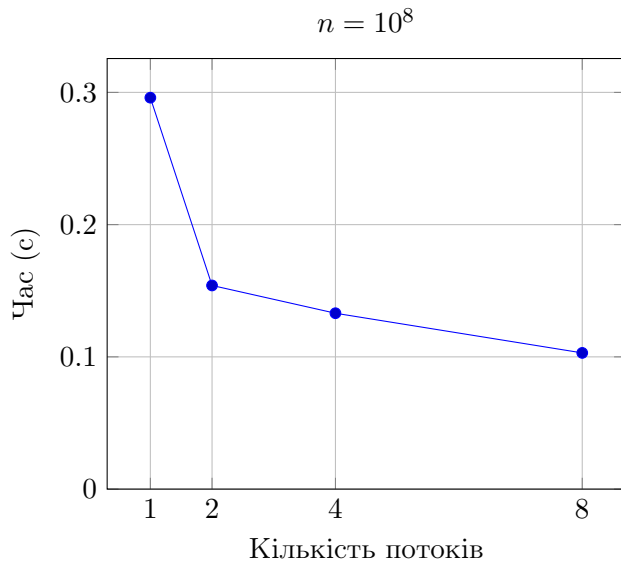
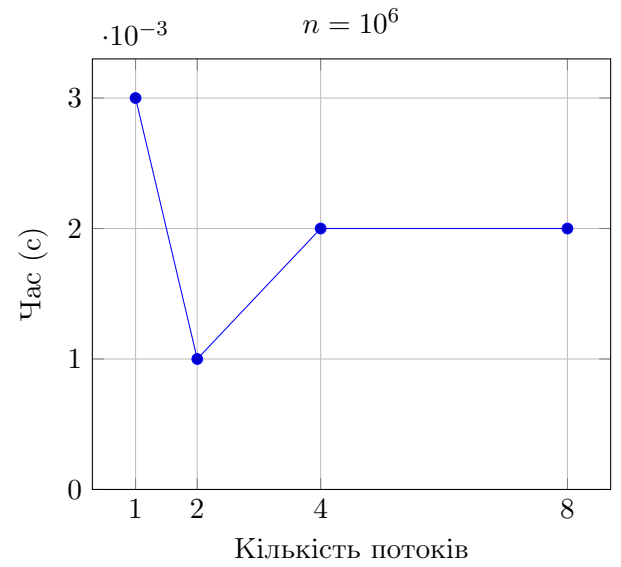
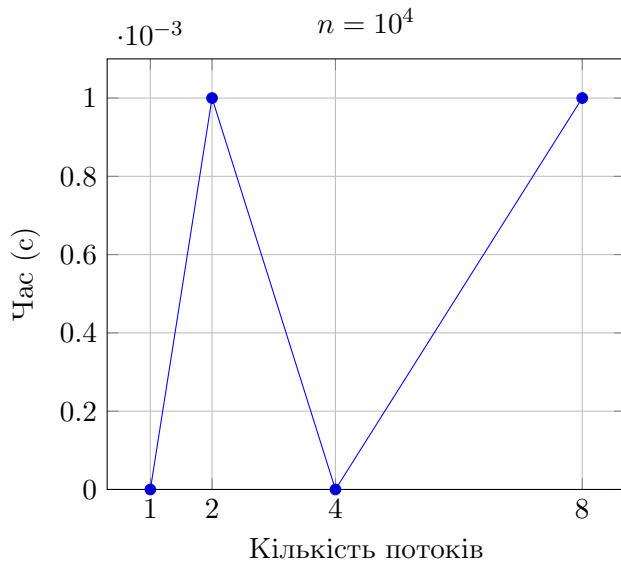


Рис. 2: Залежність часу обчислення числа π від кількості потоків для різних значень n

Обчислення проводилися з використанням компілятора GCC (MinGW) на комп'ютері з операційною системою Windows 10 та наступними характеристиками:

CPU:
 11th Gen Intel(R) Core(TM) i7-1185G7 @ 3.00GHz 1.80 GHz

Base speed: 1,80 GHz
 Cores: 4
 Logical processors: 8